

Deep Learning Recommendation Model - DLRM

Saioa Prieto Portugal, 633301

Alexander Pérez Bunea, 636393

Fabio Misino, 512377

Proyect for *SISTEMI INTELLIGENTI PER INTERNET* and *DEEP LEARNING*

Link to GitHub repository: [CODE](#)

Link to the official GitHub repository: [CODE](#)

Link to the official paper: [PAPER](#)

CONTENT

Introduction	2
Model's architecture.....	2
Architecture basics	2
DLRM architecture	3
Parallelism	3
Experiments done by Meta	4
SETUP	4
Dependencies	4
Docker	5
TESTING WITH THE MODEL.....	5
Dataset	5
Data Preprocessing.....	6
Training.....	7
Benchmarkig	8
Testing	9
CONCLUSION	10

INTRODUCTION

The evolution of recommendation systems began with content-based filtering, where experts categorized products, and users chose their preferred categories to receive suggestions. Over time, the approach shifted to collaborative filtering, which relies on past user behaviour, such as ratings, to make recommendations. Methods like neighbourhood-based approaches, which group similar users and products, and latent factor models, which use matrix factorization to uncover implicit factors that describe users and products, were successfully implemented. Although these methods have been proven to be really successful, it is only recently that they have incorporated neural networks into their techniques. This is why we are studying a Deep Learning recommendation System, model developed and studied by Meta and its affiliates.

MODEL'S ARCHITECTURE

Architecture basics

Embeddings → Embeddings map categories to a dense representation in an abstract space

- DLRM will utilize embedding tables for mapping categorical features to dense representations and then exploited via latent factor methods

Matrix factorization → Technique used in recommender systems to predict what a person might like, using only past interactions (such as ratings or clicks), using only simple matrix operations

- DLRM uses Matrix Factorization idea to calculate relations between features.

Multi-Layer Perceptron – MLP → A prediction function composed of an interleaving sequence of fully connected (FC) layers and an activation function

- DLRM will use 2 layers of MLP (top and bottom), one for transforming continuous features into dense representations and another for processing second-order interactions between features.

DLRM Architecture

INPUT:

- Users and products both are described by many of both continuous and categorical features. On one hand each categorical feature will be represented by an embedding vector, on the other hand, the continuous features will be transformed by an MLP (which we call the bottom or dense MLP) which will yield a dense representation of the same length as the categorical embedding vectors

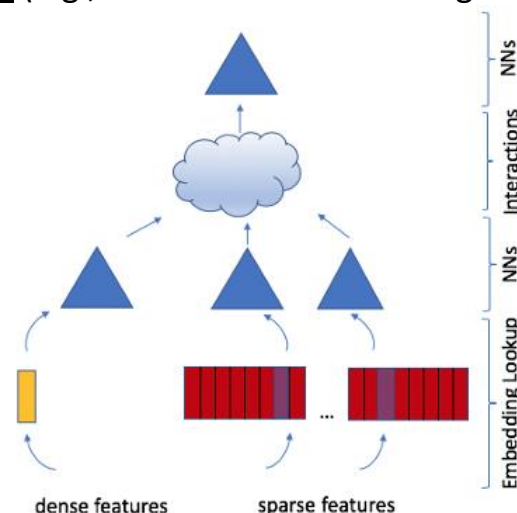
INTERACTIONS:

- Interactions between the dense representations (Second-order interactions) are calculated by making the dot product between all pairs of embedding vectors and processed dense features.

Note: Cross interactions between embeddings and MLP are calculated, and also between embeddings themselves, but not between the outputs of the MLP among themselves (because there is only one output from the MLP that represents all continuous features as a single vector).

PREDICTION:

- Then these dot products are concatenated and post-processed with another MLP (the top or output MLP), the output is passed through a sigmoid function, which provides a probability (e.g., the likelihood of clicking or not clicking on an ad).



Parallelism

DLRMs are known for having an exceptionally large number of parameters, which can lead to training times that extend over several weeks if parallelism is not employed.

Embeddings contribute the majority of the parameters because it uses several tables. Because of the size of the embeddings, applying traditional data parallelism becomes impractical.

To address this, a hybrid parallelization strategy is used: model parallelism is applied to the embedding tables, while data parallelism is reserved for the MLP components. This combination helps alleviate the memory bottlenecks caused by the embeddings and allows efficient parallelization of the forward and backward passes of the MLPs. However, frameworks like Caffe2 and PyTorch do not natively support this kind of combined parallelism. As a result, a custom implementation was developed. Custom implementation:

- Parallelism → by explicitly mapping the embedding operators to different devices.
- Communication → by using the butterfly shuffle operator, which appropriately slices the resulting embedding vectors and transfers them to the target devices

Experiments done by Meta

Meta and its affiliates, the developers of the official GitHub repository, made some experiments to compare the DLRM model with a DCN (Deep and Cross Network) model, where DLRM obtains slightly **higher** training and validation **accuracy without extensive tuning of model hyperparameters**.

Important note: official GitHub's implementations and experiments were done in a multi-GPU system with a lot of memory power.

SETUP

Dependencies

The official GitHub repositories **implement the model using both Caffe2 and PyTorch** libraries, and you can run it with either one or both.

We chose to conduct our tests with PyTorch because we have previous experience working with this library. For this reason, our setup relied heavily on the PyTorch nightly builds, which are necessary to access the latest features not yet available in the stable version. Additionally, we had to install some complementary PyTorch packages such as torchviz, torchrec-nightly, and torchx-nightly, along with some essential base libraries including numpy, scikit-learn, tqdm, pydot, and others.

All of this is powered by Python 3.10.

The **torchrec-nightly** library depends on **fbgemm_gpu**, a native library that is only available on Linux and is not supported on Windows. This dependency includes C++ and CUDA code that cannot be compiled or executed on Windows systems. To work around this limitation, we used a **Docker container with a custom image** powered by Linux. This approach offers several advantages: the environment only needs to be configured once, it ensures a complete, reproducible, and portable setup, and it eliminates the need to manually install packages each time the container is run. For creating the custom image, first we created a docker file with all the dependencies and runned it with the following command: `docker build -t torchrec_custom`. Then to use it we just runned the command to activate it adding the path of the project so it can be used once inside the docker: `docker run --gpus all -it --rm -v "your_path:/workspace" torchrec_custom bash`

TESTING WITH THE MODEL

Dataset

The dataset consists on “a portion of Criteo's traffic over a period of 7 days. Each row corresponds to a display ad served by Criteo, the target variable indicates if an ad was clicked (1) or not (0)”. It has 39 features/columns, 26 being categorical features and 13 being continuous features, and around 45 million of entries.

[illegible]

Data preprocessing

Before the data can be used by the model, it must first go through a preprocessing step. To facilitate this, the authors provide a specific Python script designed for that purpose. As a starting point, we used the `train.txt` file available from the official dataset website.

The preprocessing script splits the data into “days” and processes each day separately. It reads the entire dataset, divides it evenly across a number of days, and ensures that each day contains a minimum of approximately 1.28 million entries. For each split, the program reads all entries, records whether each is labeled as 1 or 0, and stores the result in a file named *train_day_X.npz*. At the end of this process, it attempts to merge all the generated sub-files into a single output file called *kaggleAdDisplayChallenge_processed.npz*.

We attempted to preprocess the full dataset of 45 million entries; however, after 12 hours of processing, the program was terminated/killed during the final merging step due to resource exhaustion. After several attempts, we found that **the training process requires a minimum of 7 "days" to complete successfully** without errors. To meet this constraint while avoiding memory issues, **we sampled 9 million entries from `train.txt`** using the `head` command, redirecting the output to a new text file, and processed them. This sample size is roughly sufficient to span the required 7 days.

To run the preprocess, you must enter in the bash the following code:

```
python
```

```
import os
```

```
from data_utils import getCriteoAdData
```

```
getCriteoAdData(
```

```
    datafile="/workspace/dlrm/datasets/train_9M.txt",
```

```
    o_filename="kaggleAdDisplayChallenge_processed",
```

```
    max_ind_range=9000000,
```

```
    sub_sample_rate=0.0,
```

```
days=7,  
data_split='train',  
randomize='total',  
criteo_kaggle=True,  
memory_map=False  
)
```

TRAINING

After obtaining all the pre-processed data, we proceeded to the next step: training. As mentioned earlier, the model is extremely resource-intensive. We attempted numerous training configurations, but all of them resulted in the process being killed due to excessive resource usage. On the command used to execute the training the "light" parameters we experimented with in an effort to reduce the load can be seen:

```
python dlrn_s_pytorch.py \  
--arch-sparse-feature-size=8 \  
--arch-mlp-bot="13-8-8" \  
--arch-mlp-top="8-4-1" \  
--data-generation=dataset \  
--data-set=kaggle \  
--raw-data-file=./datasets/train_9M.txt \  
--processed-data-file=./datasets/kaggleAdDisplayChallenge_processed.npz \  
--loss-function=bce \  
--round-targets=True \  
--learning-rate=0.05 \  
--mini-batch-size=1 \  
--test-mini-batch-size=2 \  
--test-num-workers=0 \  
--num-batches=10 \  
--print-freq=10 \  
--print-time \  
--test-freq=10 \  

```


`--save-model=dlrm_model.pth`

Important note: We perform an accuracy check (if it's better than the previous training, it gets saved) so the trained model can be stored in first place and after if a training with better results occurs it also saves it.

TRAINING OUTPUT:

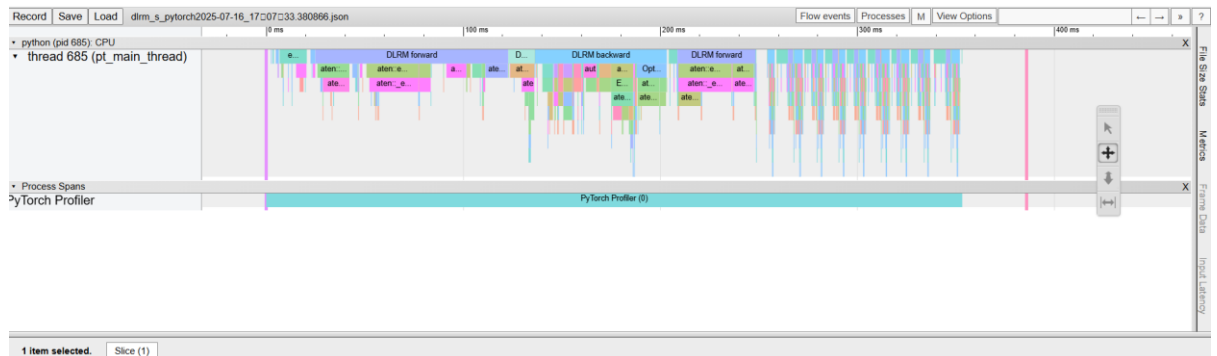
The training process completed 10 iterations for epoch 0, with an average iteration time of approximately 228.55 ms. The final **loss** at the end of this epoch was 0.675228, indicating the model is starting to learn meaningful patterns from the data. After testing, the model achieved an **accuracy** of 90.000%. The trained model at this stage was saved to the file *dlrm_model.pth* for future use or further training.

BENCHMARKING

To perform benchmarking, we used the code specifically designed for this dataset, which is also provided in the official repository. The benchmarking was **executed on both CPU and GPU**, each generating its own set of output files for result evaluation. The **output files include .prof and .json formats**. These files are generated by the profiler—a tool that measures code performance, timing, and resource usage. They contain detailed technical data used for in-depth performance analysis.

OUTPUT FILES:

- Two **.prof files** → It includes detailed performance metrics for both CPU and CUDA (GPU) operations.
- One **.json file** → It contains both technical information about the GPU where the code was executed and a list of execution events, each representing an operation on the CPU or GPU during training or inference.
 - It can be visualized in: <chrome://tracing>



1 item selected. Slice (1)	
Title	DLRM loss compute
Category	user_annotation
User Friendly Category	other
Start	122,912 ms
Wall Duration	13,285 ms
Self Time	1,047 ms
Args	
External id	185
Ev idx	184

- One **.log file** → contains the output of the program, in this case a training

Important note: When running the benchmark, an error appears in the interface indicating that the .prof and .json files cannot be found, as they are not created in the same directory as the program. However, the files are indeed generated correctly and safely stored in a different location.

Command for perform the benchmark: `./bench/dlrm_s_benchmark.sh "--enable-profiling --save-model=dlrm_model.pth"`

TESTING

While using the official test script, we encountered an error with the file. This issue arised because the ***dlrm_s_test.sh*** file **was created or edited on a Windows system** and contains Windows-style line endings (^M), which are incompatible with Unix/Linux environments. As a result, the script fails to execute properly. The solution is to convert the file to Unix format using the command `dos2unix`

`dlrm_s_test.sh`. It is important to note that `dos2unix` must be installed within the Docker container for this to work.

In addition to resolving the script issue, we also **preprocessed the test** data by setting the parameter `data_split='test'`, which generated new files specifically for the inference step. However, despite following the steps outlined in the official README, we encountered several problems related to data files and dataset size. Furthermore, **the official test script was not usable in our setup**, as we were not working with the full official test dataset. Instead, we used a manually created 9-million-entry sample subset to ensure the code could run successfully within our hardware limitations.

Finally, since we were unable to use the official test scripts and ran into issues when attempting our own inference, we decided not to perform testing.

The main obstacle was that any change or fix required reprocessing the data—a task that took at least 21 minutes each time. This made it impractical to quickly iterate or troubleshoot problems. As a result, the long preprocessing time became a bottleneck, preventing us from efficiently resolving the issues and completing the inference step.

CONCLUSION

In conclusion, our work highlights both the challenges and the potential of using artificial intelligence in recommendation systems. While implementing the model, we faced several difficulties—most notably, the lack of clear documentation on how the implementation code works, and numerous issues stemming from the data preprocessing process.

Despite these challenges, the simplicity of the model and its strong overall performance demonstrate the value and promise of AI in this field. These positive results reinforce the idea that investing in better tools, clearer documentation, and more streamlined workflows can significantly accelerate the adoption and impact of AI-driven recommendation technologies.